

QUESTIONS DE RÉFLEXION :

SOMMAIRE :

- **I:** Installer et configurer son environnement de travail
[page 2]
- **II:** Maquetter des interfaces web ou web mobile
[page 4]
- **III:** Réaliser des interfaces utilisateur statiques web ou web mobile
[page 5]
- **IV:** Développer la partie dynamique des interfaces utilisateur web ou web mobile
[page 8]

I: Installer et configurer son environnement de travail

L'éditeur de code (IDE) :

J'ai choisi d'utiliser Visual Studio Code (couramment nommé VSCode) principalement car déjà il marche bien sur mon installation Linux.

Il intègre aussi Emet, dont ses raccourcis sont très pratiques pour gagner du temps lors du développement d'un site.

Le support d'extensions, surtout celles liées au développement Web dans mon cas, est un bon argument pour développer avec cet éditeur.

Et le fait de pouvoir ouvrir des terminaux depuis l'éditeur de code directement est très agréable. Combiné au système des Tasks, ce tout me permet de créer un workflow efficace qui me convient amplement.

Les navigateurs de test :

J'ai utilisé deux navigateurs différents pour tester le site: Brave, qui est Chromium based. Et Firefox Desktop/Mobile, qui sont tous deux Gecko based.

Je peux alors voir si le site marche comme attendu sur deux navigateurs principaux, qui utilisent deux technologies différentes.

Firefox, car c'est mon navigateur par défaut que j'utilise quasiment tout le temps. Et Brave, car la console de développeur est plus développée comparé à celle de Firefox. Et aussi car il supporte très bien les Progressive Web App (PWA). Ce qui est essentiel pour développer le projet. En plus d'être le meilleur navigateur basé sur Chromium que je connaisse.

Les outils utilisés pour le débogage :

On a d'abord un indispensable: La console de développeur. Pour debug les problèmes de style, de JavaScript, ou autres soucis variés surtout.

Pour vérifier que le site est bien une PWA fonctionnelle, comme Firefox Desktop a un support peu développé de cette technologie, je regarde si tout marche bien sur ce point sur Brave. Qui lui n'a pas ce soucis.

Et pour tester le responsive, l'extension "Mobile First" me permet de simuler l'affichage du site sur un appareil mobile, comme sur une tablette ou sur un ordinateur. Sans devoir

changer la taille du navigateur, ou celle de la console. Ce qui est très pratique.

II: Maquetter des interfaces web ou web mobile

L'organisation de l'écran :

Pour rendre l'application adaptée à un usage mobile, cette dernière a été pensée pour suivre un design en flex layout, la rendant responsive pour adapter la taille et la position des éléments de l'interface selon les dimensions de l'appareil (dont celui du téléphone).

Avec un header distant du reste de l'interface. Et une volonté de centrer le plus d'éléments possible pour les rendre les plus facile d'accès par l'utilisateur.

Si il y a une action proposée, elle suit souvent le même schéma: Le bouton associé se trouve tout en bas de l'écran. Pour un usage mobile, il est facile d'accès sans trop être si difficile à presser.

Finalement, les actions plus sensibles se trouvent intégrées dans le burger menu du header. Pas besoin de fouiller dans des menus obscurs pour supprimer ses données.

La navigation :

Comme sur un site classique, l'icône du site sur les pages redirige vers celle de base de ce dernier . En une pression de cette dite icône, il est possible de quitter la page qui affiche tous les IMC calculés enregistrés par exemple.

Les boutons symbolisant une action permettent de naviguer entre les pages du site de manière claire.

La hauteur des pages ne dépassent généralement pas la hauteur du navigateur, hormis celle qui affiche tous les IMC calculés dans une forme de liste datée. Retrouver un IMC est alors facile: l'utilisateur scroll dans la liste, en se repérant avec les dates pour trouver ce qu'il cherche.

La taille des zones interactives :

Les boutons symbolisant une action ont été pensés pour être assez grands et visibles pour être facilement identifiés et interactifs.

III: Réaliser des interfaces utilisateur statiques web ou web mobile

La lisibilité :

Pour rendre le texte le plus lisible possible, il a fallu que leur couleur ait un contraste assez fort avec les différents arrière-plan du site. Pour le fond en dégradé:

Contrast Checker

[Home](#) > [Resources](#) > Contrast Checker

Foreground

Hex Value
#22181C

Color Picker

Alpha
1

Lightness

Background

Hex Value
#99BDFF

Color Picker

Lightness

Contrast Ratio
9.11:1

[permalink](#)

Normal Text

WCAG AA: **Pass**
WCAG AAA: **Pass**

The five boxing wizards jump quickly.

Large Text

WCAG AA: **Pass**
WCAG AAA: **Pass**

The five boxing wizards jump quickly.

Graphical Objects and User Interface Components

WCAG AA: **Pass**

★
Text Input

Contrast Checker

[Home](#) > [Resources](#) > Contrast Checker

Foreground

Hex Value
#22181C

Color Picker

Alpha
1

Lightness

Background

Hex Value
#B4FFE5

Color Picker

Lightness

Contrast Ratio
15.11:1

[permalink](#)

Normal Text

WCAG AA: **Pass**
WCAG AAA: **Pass**

The five boxing wizards jump quickly.

Large Text

WCAG AA: **Pass**
WCAG AAA: **Pass**

The five boxing wizards jump quickly.

Graphical Objects and User Interface Components

WCAG AA: **Pass**

★
Text Input

Contrast Checker

[Home](#) > [Resources](#) > Contrast Checker

Foreground

Hex Value
#22181C

Color Picker

Alpha
1

Lightness

Background

Hex Value
#E298FB

Color Picker

Lightness

Contrast Ratio
8.32:1

[permalink](#)

Normal Text

WCAG AA: **Pass**
WCAG AAA: **Pass**

The five boxing wizards jump quickly.

Large Text

WCAG AA: **Pass**
WCAG AAA: **Pass**

The five boxing wizards jump quickly.

Graphical Objects and User Interface Components

WCAG AA: **Pass**

★
Text Input

Pour le fond orange des boutons :

Contrast Checker

[Home](#) > [Resources](#) > Contrast Checker

Foreground

Hex Value
#22181C

Color Picker

Alpha
1

Lightness

Background

Hex Value
#FFA666

Color Picker

Lightness

Contrast Ratio
8.96:1

[permalink](#)

Normal Text

WCAG AA: **Pass**
WCAG AAA: **Pass**

The five boxing wizards jump quickly.

Large Text

WCAG AA: **Pass**
WCAG AAA: **Pass**

The five boxing wizards jump quickly.

Graphical Objects and User Interface Components

WCAG AA: **Pass**

Et pour le contraste avec le fond blanc, les deux couleurs sont plus que compatibles.

La taille du texte a été définie en CSS avec l'unité de mesure "rem", avec une taille de base assez grande pour être facilement vu et lu par l'utilisateur (15px mis dans la balise html, et 10px en responsive mobile).

Toutes ces couleurs ont été conservées dans un fichier CSS dédié. La syntaxe ":root {}" m'a permis de créer des variables qui contiennent ces couleurs.

La cohérence visuelle :

Ici tout se passe au niveau du CSS. Pour les couleurs déjà, l'utilisation de ":root {}" est très pratique. Car ces variables créées peuvent être utilisées partout si le fichier CSS est chargé. Idem pour la typographie, les polices utilisées existent dans un fichier dédié, en compagnie de la configuration des balises de texte. Pour les utiliser, faut juste utiliser les variables dans le :root de ce fichier CSS. Pour les boutons, tout le site obtient ses couleurs depuis le fichier CSS des couleurs.

Ce tout permet d'avoir les mêmes couleurs, polices, tailles de texte partout sur l'application.

L'adaptation à différentes tailles d'écran :

Comme énoncé plus tôt, le site étant pensé en flex layout cette adaptation est plutôt simple à mettre en place.

L'élément qui change le plus dans l'application est le header.

Sur mobile et tablette, elle se trouve tout en haut de l'écran. Ici le layout est en flex, flex-direction: column pour permettre un empilement vertical optimisé pour un usage mobile. Tandis que sur un affichage Desktop, ce même header se trouve sur le côté gauche de l'écran. Là, le flex-direction a été défini sur row. Dans les deux cas, l'espace entre le header et le reste des pages est assez large. Sans compter que sur Mobile, il prend bien moins de place que sur Desktop.

IV: Développer la partie dynamique des interfaces utilisateur web ou web mobile

Le calcul de l'IMC :

Ce calcul se passe dans le fichier JavaScript de la page dédiée à cette fonctionnalité.

Dès que le formulaire qui demande d'entrer des données pour l'opération est correctement rempli, la fonction `calculateIMC()` est appelée. Les valeurs (taille et poids) dans les inputs sont récupérés, convertis en Numbers

EXEMPLE:	<pre>let weight = Number(calculateImcFormWeightInput.value);</pre>
----------	--

Puis ils sont utilisés dans la formule $poids / (taille * taille)$
L'IMC calculé est enfin arrondi au centième près (2 zéros après la virgule maximum), avant d'être retourné à l'endroit où la fonction de calcul a été appelée.

L'enregistrement des mesures :

Un script JavaScript a été créé, pour agir comme un Manager pour le localStorage. C'est le Local Storage Manager, intérieurement au projet.

Avant d'expliquer comment l'IMC calculé est sauvegardé, il faut comprendre la structure des données.

Le Manager conserve les données dans le format JSON. Il crée des listes d'Objets, qui contiennent trois informations:

- L'IMC Calculé
- Quand ce calcul a eu lieu (date et heure qui est générée automatiquement lors de l'enregistrement)
- Et un commentaire court, lié à l'IMC calculé

Le Manager peut gérer un nombre infini de listes de 25 éléments au total, et pas plus.

Dès qu'un IMC a été calculé, la fonction `updateLocalStorage()` est appelée, avec en argument une donnée, une nouvelle instance de l'Objet décrit plus tôt qui va être créée.

La première étape est de savoir où ranger cette donnée, les listes de données dans le localStorage sont toutes indexées comme dans une liste classique. On veut savoir l'index d'une liste disponible (qui a moins de 25 éléments dedans). La

fonction `lookForAvaibleFileIdx()` joue ce rôle, elle renvoie cet index. Elle itère dans toutes les entrées du `localStorage`, parse en JSON le texte pour vérifier le nombre d'éléments. L'index renvoyé peut pointer vers une liste qui a encore de la place disponible pour une nouvelle donnée. Mais il peut aussi pointer vers rien du tout (l'index est supérieur à la longueur du `localStorage` + 1).

Cet index est ensuite envoyé vers la fonction `writeJSONFile(index, data)`. Où la donnée `data` passée dans `updateLocalStorage(data)` est passée. En plus de l'index trouvé.

Cette fonction écrit une liste dans le `localStorage`, si la longueur du `localStorage` est supérieure ou égale à l'index + 1, on met la liste à l'index donné à jour de la manière suivante: Une nouvelle liste qui contient en premier élément la nouvelle donnée, puis qui contient tous les éléments de la liste que l'on veut mettre à jour. Sinon, une nouvelle liste est créée à l'index donné, juste avec la donnée uniquement. Les données les plus récentes vont toujours au début des listes dans le `localStorage`. En somme, les données sont écrites à l'envers. De la dernière à la première.

L'affichage de l'historique :

Le Local Storage Manager est aussi utilisé ici.

La fonction `getData()` permet de renvoyer une liste de données. Elle demande un argument optionnel qui limite le nombre de données envoyées par le Manager.

Premièrement, elle récupère toutes les données de toutes les listes du `localStorage`. De la toute dernière à la toute première. Créant la variable `baseData`.

Dans laquelle on va itérer x nombre de fois. Ici x peut être égal à la longueur de `baseData` (qui est une liste aussi), ou si le nombre de données que l'on veut est supérieur à 0 (avec l'argument optionnel cité plus tôt), x sera égal à l'argument optionnel si il ne dépasse pas la longueur de `baseData`. Sinon x sera égal à cette longueur. Ceci peut vite être fait avec une seule ligne de code.

```
if (maxEntries > 0) { iter = Math.min(Math.max(maxEntries, 0), baseData.length); }
```

Puis on renvoie une liste qui contient x nombre de données. Des dernières enregistrées aux premières.

Faut plus que le script qui demande ces données les utilise quand elles sont reçues. Les deux pages qui en demandent sont l'index, qui en veut que 5, les 5 dernières. Et celle qui liste toutes les IMC calculées. Elle veut toutes les données. Dans le premier cas, la fonction est appelée ainsi: *getData(5)* . Dans l'autre, juste *getData()* .

De l'HTML est créé en fonction des données récupérées, et inséré dans le DOM des pages. Vu que des id et class sont déjà mis dans le HTML à insérer, le style s'applique tout seul.